

Rapport : Gestion de fichiers dans la mémoire secondaire

1. Objectif du Code

Le programme a pour objectif de simuler un système de gestion de fichiers basé sur une mémoire secondaire. Il permet à l'utilisateur de créer, manipuler, rechercher, supprimer et gérer des fichiers en utilisant différentes méthodes d'organisation globale (contiguë ou chaînée) et interne (triée ou non triée). Ce système offre une interface interactive pour effectuer des opérations courantes tout en mettant l'accent sur la gestion efficace de l'espace mémoire.

2. Conception

2.1. Architecture Générale

Le programme repose sur trois concepts principaux :

1. **Mémoire Secondaire :**
 - a. Simulée comme un tableau de blocs.
 - b. Chaque bloc peut être libre ou occupé, et il contient des données associées aux fichiers.
2. **Fichiers :**
 - a. Représentés par des structures contenant des métadonnées (nom, taille, mode d'organisation) et une liste de blocs utilisés.
3. **Interface Utilisateur :**
 - a. Un menu interactif est présenté dans une boucle do-while permettant d'exécuter diverses fonctionnalités.

2.2. Structures de Données

- **Mémoire Secondaire :** Chaque bloc est représenté par une structure qui contient un indicateur d'occupation et un tableau pour stocker les données :

```
typedef struct {  
    int est_occupe; // Indique si le bloc est libre (0) ou occupé  
(1)  
    char contenu[256]; // Contenu stocké  
} Bloc;
```

- **Fichiers :** Les fichiers sont structurés de la manière suivante :

```

typedef struct {
    char nom[50]; // Nom du fichier
    int nb_enregistrements; // Nombre total d'enregistrements
    char mode_globale[10]; // Mode d'organisation globale : contigu
ou chaîné
    char mode_interne[10]; // Mode d'organisation interne : trié ou
non trié
    int blocs[100]; // Liste des indices des blocs utilisés
    int nb_blocs_utilises; // Nombre de blocs actuellement utilisés
} Fichier;

```

3. Fonctionnalités Implémentées

3.1. Initialisation de la Mémoire Secondaire

- Permet de configurer la mémoire avec un nombre défini de blocs.
- Réinitialise tous les blocs comme libres.

3.2. Création de Fichiers

- Crée un fichier en spécifiant son nom, son mode d'organisation, et le nombre d'enregistrements.
- Les blocs nécessaires sont alloués en fonction du mode global. Selon le mode choisi (contigu ou chaîné), les blocs seront alloués soit de manière continue, soit chaînés entre eux.

3.3. Chargement de Fichiers

- Charge un fichier existant pour effectuer des manipulations ou des recherches. Cette fonctionnalité permet de charger un fichier en mémoire afin de l'utiliser dans d'autres opérations comme l'insertion ou la recherche.

3.4. Manipulation des Fichiers

1. **Insertion d'Enregistrements :**
 - a. Insère des données dans un fichier en utilisant les blocs disponibles.
L'allocation des blocs dépend du mode d'organisation du fichier.
2. **Recherche d'Enregistrements :**
 - a. Recherche un enregistrement par son ID dans les blocs du fichier, permettant de retrouver rapidement des données.

3. Suppression :

- a. **Logique** : Marque un enregistrement comme supprimé sans effacer les blocs physiques.
- b. **Physique** : Libère complètement les blocs utilisés en supprimant les données associées.

3.5. Gestion Avancée

- **Compactage** : Regroupe les blocs occupés d'un fichier pour réduire les trous mémoirels et améliorer l'utilisation de l'espace.
- **Défragmentation** : Réorganise tous les blocs de mémoire pour optimiser leur utilisation et minimiser les espaces non alloués.
- **Renommage** : Permet de changer le nom d'un fichier existant.
- **Vider la Mémoire Secondaire** : Supprime tous les fichiers et libère tous les blocs, réinitialisant ainsi l'état de la mémoire secondaire.

4. Résultats Obtenus

4.1. Tests et Observations

- **Test 1 : Initialisation**
 - **Entrée** : Mémoire configurée avec 50 blocs.
 - **Résultat** : Tous les blocs sont initialisés comme libres.
- **Test 2 : Création de Fichier**
 - **Entrée** : Nom = "Fichier1", Enregistrements = 10, Mode = "contigu", "nontrié".
 - **Résultat** : Fichier créé avec 10 blocs alloués successivement.
- **Test 3 : Insertion d'Enregistrements**
 - **Entrée** : ID = 1, Données = "Record1".
 - **Résultat** : Données insérées avec succès dans le bloc disponible.
- **Test 4 : Recherche d'Enregistrements**
 - **Entrée** : ID = 1.
 - **Résultat** : Enregistrement trouvé dans le bloc assigné.
- **Test 5 : Défragmentation**
 - **Entrée** : Mémoire fragmentée.
 - **Résultat** : Mémoire réorganisée pour réduire les trous entre les blocs.

5. Analyse et Améliorations

1. Points Positifs

- Le code implémente efficacement les concepts de gestion de fichiers et de mémoire secondaire.
- Les fonctionnalités essentielles sont bien couvertes et fonctionnent comme prévu.

6. Conclusion

Le projet offre une solution fonctionnelle pour simuler la gestion de fichiers dans une mémoire secondaire. Bien que le code soit adapté à des fins pédagogiques, des améliorations peuvent le rendre plus robuste, performant et adapté à des applications réelles. La gestion des fichiers et de la mémoire secondaire est efficace, mais une optimisation de certains aspects pourrait améliorer considérablement l'expérience utilisateur et les performances globales du système.